BPM and linearity characterization
Mathematics and implementation, version 1

Wenu-imaging

10 November 2026

Executive summary

This recipe measures detector response and produces a candidate bad-pixel mask from paired flat exposures, an existing master bias and an admitted prior mask. Linearity characterization is measurement only: it does not correct science images or change the gain. Cross-talk is outside its present scope.

For each detector, native C services perform row scan correction, master-bias subtraction, trimming and ROI/direct-site measurements, then characterize the candidate BPM using the same fixed calibrations. Detectors run serially. Final MEF assembly preserves the detector image payloads and joins their native ASDF measurements, input identities and calibration HDU bindings.

Project TOML defines the directory roots. Raw observations remain read-only; recipe controls belong in orchestration; products, logs and work belong below reduced. Separate detector masters retain their own provenance. Shared calibrations collect detector bindings only when their metadata agree.

The final products are FITS, ASDF and a log, with frozen configuration and input-preservation evidence. A zero mask value means no detected reason, not certification. Mac qualification and exact CCD65 comparison passed for the single-detector native coordinator; the extension for separate calibration files requires its focused Mac check before the full 32-detector run.

Scope and configuration

This guide describes the preparation kernels and their FITS/recipe/ASDF integration. The numerical kernels and native publication passed their Mac gates. The CCD65 pilot passed exact saved-payload/direct-sample checks and visual plot inspection. The native coordinator passed its focused Mac gate and exact CCD65 comparison; physical calibration acceptance is separate. The reviewed design is in the developer design directory:

`linearity-preparation-overscan-asdf-v1.md`.

The detector TOML describes the full detector bands and dimensions. The processing TOML selects the sample columns, trim rectangle, rejection counts, ROIs and preparation controls. The shared Python resolver validates their relationship and freezes both original texts and SHA256 identities. There is no PAF fallback. For CCD65, the full overscan is columns 2097–2144, but the sample is 2100–2142. These distinctions survive in the resolved record.

Exact row correction and arithmetic

For raw frame f, raw row y, and the n enabled scan values sorted in ascending order, the existing trimmed-mean routine returns

$$O_f(y) = \mathrm{pix}\left[\frac{\sum_{k=l+1}^{n-h} v_{f,y,(k)}}{n-l-h}\right]. \tag{1}$$

The sum and division use double precision; pix converts to the native `pixelvalue` type. Low/high rejection is $l = h = 10$ in the selected OmegaCAM profile: 43 reads retain 23, and 44 retain 24. No sigma clipping, smoothing, row fitting, endpoint extrapolation or global-level replacement is introduced. First and last raw rows use their own scan samples. The active-pixel BPM is not applied to electronic scan reads.

The native stages are

$$U_f(x,y) = \mathrm{pix}[D_f(x,y) - O_f(y)], \tag{2}$$

$$S_f(p) = \mathrm{pix}[U_f(p) - B(p)] \quad \text{(flats only)}. \tag{3}$$

The trim uses inclusive FITS coordinates after scan subtraction. An active row j maps to raw row $j + y_{\min} - 1$. Preserve both native roundings: do not merge the subtractions. A supplied master is already scan-corrected and active; it is subtracted once and receives no second scan correction. Bias-role frame preparation has no master subtraction. The existing bias service performs frame admission and stacking; this increment does not change its median or rejection policy. The recipe verifies master configuration, source memberships and lineage before reuse; compatibility is not inferred from pixel values.

Corrected ROI statistics and populations

The paired service first reuses the unchanged raw-pair measurement service. For each configured ROI it uses the common population $\mathcal{P}$ admitted by the declared BPM and finite/raw-saturation checks in both flats and both raw biases. The raw saturation boundary remains 65534 ADU. Pair members retain their original identity, HDU, input index and exact duration.

For corrected member values S_{1p}, S_{2p}, let $N = |\mathcal{P}|$. The services retain ordinary means and sample scatter of each member and their difference:

$$\overline{S}_i = N^{-1} \sum_{p \in \mathcal{P}} S_{ip}, \tag{4}$$

$$s_i^2 = (N-1)^{-1} \sum_{p \in \mathcal{P}} (S_{ip} - \overline{S}_i)^2, \tag{5}$$

$$d_p = \mathrm{pix}[S_{1p} - S_{2p}], \qquad s_d^2 = (N-1)^{-1} \sum_{p \in \mathcal{P}} (d_p - \overline{d})^2. \tag{6}$$

Difference pixel arithmetic follows `image_pair_stats_checked`. The native result stores standard deviations and availability; the native TSV transport stores their squared values and $s_d^2/2$ with explicit definitions. For $N < 2$, sample scatter is unavailable, not zero. A corrected nonfinite value cannot silently remove an otherwise admitted measured pixel: all three corrected population counts must equal the raw count.

Spatial scatter and difference scatter are diagnostic quantities, not assigned independent weights. The shared master cancels algebraically in differences, subject to native rounding. Raw bias-only RON and raw gain diagnostics retain their original definitions. No uncertainty model or physical response is estimated. ROI means remain distinct from the direct physical pixel samples required by the later nonlinear model.

Connection to implementation and verification

`pycavi_linearity_config.resolve` performs TOML resolution only.

`mef_linearity_prepare_frame` validates geometry and finite enabled scan reads, copies the borrowed raw image and uses the existing `mef_overscan_correction_traced` arithmetic. Its optional trace copies each estimate before QC median sorting, preserving physical row order. The legacy interfaces pass a null trace and keep their numerical behavior. The result owns the active image, trace and settings copy.

`mef_linearity_prepare_pairs` corrects the two biases for trace evidence and two flats at a time, reusing raw-pair admission and the existing masked single-image and difference statistics. Results own copied identities, raw diagnostics, corrected ROI statistics and traces. Borrowed inputs stay unchanged; occupied outputs, invalid settings, resource limits and failures do not expose partial completed owners. Custom allocation checks cover the new owners and snapshot owners; they do not establish general Eclipse allocation safety.

Independent ramp fixtures cover both scan widths, raw/active edges, exact singleton-path agreement, common-population analytical moments, and persistent owner-allocation failures. Existing bias-service, sample and raw-pair controls check affected shared paths. Six TOML and five footprint controls check the configuration boundary. Agent Linux success is not actual Mac acceptance or scientific validation. The actual Mac kernel gate passed four native and eleven Python controls; new integration qualification remains separate.

Recipe and immutable ASDF boundary

`pycavi_linearity_recipe.preparation_dependencies` reads the supplied TOMLs and preserved sample/master QC graph. It requires exact scan, trim and stack compatibility with the master snapshot and matches original identities, HDU bindings, accepted/rejected memberships and readout. Legacy header fields travel through a generated native compatibility TOML; they are not scientific configuration overrides. Historical bias QC may have null image digests; the completed bias execution receipt then supplies those fingerprints explicitly.

`prepare_detector` fingerprints input images before native processing, checks them again afterward, and freezes metadata and executable identities. `mefpp_linearity_prepare_command` reads every bias for ordered traces, then streams flat pairs through `mef_linearity_from_files`, using the qualified preparation service and the existing direct-pixel extraction. It reuses the existing master and never rebuilds it. Direct-pixel values are replay checks against the preserved archive, not substitutes for full ROI measurements. No separate singleton extraction campaign is required.

The recipe envelope references a project TOML in orchestration. That project owns the raw, calibration, output, work and log roots. Observations remain read-only in raw; existing calibrations are read-only dependencies below reduced/cal. Controls and metadata requests stay in orchestration, and all products, working files, logs and execution receipts stay below reduced.

With `publication = "native"`, the command retains only small ROI metadata between pairs. `mef_linearity_archive_add_pair` copies the already measured means and variances before the pair's image owners are released. `mef_linearity_archive_table` restores original flat-list and configured ROI order and writes the exposure table exclusively. The shared `mef_product_qc_write` validates and publishes ASDF using the same qualified C library as bias and BPM. This is an explicit table product: application schema 2.0.0, ASDF standard 1.6.0, no invented image HDU. Existing MEF and primary-image publication retain their separate layout checks.

The native ASDF contains typed exposure measurements, input identities, original indices and source HDUs, calibration bindings and the exact native transports. Those transports preserve raw diagnostics, pair differences, populations, ordered row-correction traces and direct-site samples without repeating an estimator. All biases are used for scan traces; their separate accepted/rejected membership in the earlier master remains explicit in the source-binding and trace evidence. The native context declares unavailable absolute noise/covariance and an unperformed physical fit. No linearity or cross-talk correction is applied.

Python admits and freezes existing dependency metadata, copies the preserved QC graph and verifies the completed native archive. It does not complete measurements or write the new ASDF. The former Python publisher remains available explicitly as a compatibility/reference path. Older recipe envelopes without a publication field retain that path; the new example selects native publication.

`pycavi_linearity_native_archive.read_product` checks the local schema, producer, configuration, detector/input/calibration bindings, frozen relative dependency graph and equality of typed measurements with saved native transports. Relocation needs neither raw images nor the native working directory. The modelling accessor refuses to imply that missing uncertainty inputs are ready for a fit. Original archives remain immutable, and outputs cannot be reused or overwritten. Native preparation publication and its complete recipe envelope require the focused Mac gate; observational acceptance remains pending.

Independent tiny FITS tests cover shuffled HDUs, known row ramps and ROI moments, all ten bias memberships, original direct-site values, duplicate and resource refusals, source preservation, ASDF roundtrip and relocation without native files. Focused file-adapter allocation controls and the unchanged singleton command check shared interfaces. None is scientific acceptance of observational calibration or a general dependency memory-safety claim.

Combined detector characterization

The combined recipe uses these entry points:

`pycavi_bpm_linearity_plan.admit`

`pycavi_bpm_linearity_recipe.execute`

One TOML references the project roots, qualified native executables, detector/processing TOMLs and each detector's preserved sample/master dependencies. BPM acquisition limits, output mode and named reference ROIs are explicit; the coordinator supplies no new scientific defaults. The source configuration and coordinator implementation hashes are retained in the joined manifest.

All dependency metadata is admitted before image processing. Each detector is processed serially: native linearity preparation first, then native BPM characterization. Both use the admitted prior mask and master. The newly derived BPM remains a candidate; it is never fed back into the same measurements. The native routines retain their estimators, masks, populations and array arithmetic. The coordinator performs no pixel computation or ASDF writing. Native product companions remain together; process console logs use the project log root. Execution requires niceness at least 10.

`BatchInputs` fingerprints each unique image once before and once after the batch. Individual stages check file identity, size, modification time and mode against that frozen baseline. The final byte digest is checked even when those metadata fields are unchanged. This avoids rehashing shared multi-extension raw files for every detector. Child receipts explicitly defer batch preservation; the completed parent manifest resolves that check. Native source/binary and configuration identities are checked independently.

The existing input-plane ceiling is checked for the BPM stage from the declared raw geometry, flat count and two active calibration planes. This is a loaded-input budget, not an RSS claim or a new bound on the existing kernel's scratch owners. One native operation and one detector are active at a time; the coordinator holds no detector cubes. The linearity service retains its existing working-pixel limits.

The manifest is published exclusively after source verification. Without an assembly section its scope remains `detector_characterization`, with final MEF state `not_assembled`. With an explicit assembly executable, source revision and reference detector, the scope is `bpm_linearity` and final MEF state is `assembled`. Scientific acceptance remains false. The native assembler requires common ordered flat inputs. Each detector retains its admitted master and prior mask; separate calibration files are supported.

Final BPM MEF and QC publication

The standalone assembly command is `mefpp_bpm_assemble`; the native recipe reuses its shared C services in process. It consumes only the completed detector FITS/ASDF artifacts; it never reads raw frames or repeats characterization. The routine `mef_bpm_mef_assemble` shares the bias assembler's bounded byte copier and exclusive sibling-file publication. The original bias one-extension contract remains separate.

The final primary has no image data and retains the explicitly chosen reference detector's metadata. Detector masks occupy HDUs $1, \ldots, N$ in catalogue order. All diagnostic planes then follow in detector and original-HDU order: coverage, sample count, new flags, selection, and the available prior flags or selection. Each extension records `DETID` and `EXTVER`=configuration slot+1. Mask `EXTNAME` is the detector ID; diagnostic names retain their plane role. Pixel encodings, geometry and every padded data byte are copied unchanged. The final basename replaces `ORIGFILE`; stale header checksums are removed, while data checksums remain valid for unchanged image payloads.

The routine `mef_bpm_assembly_qc_write` uses the qualified C ASDF mapping interface. It joins product measurements, input participation, per-input measurements and calibration HDU bindings, without recalculating any value. Calibration records are joined by their original file IDs in first-encounter order. Shared IDs require identical metadata and collect detector bindings; distinct masters or masks remain separate records with their original source HDUs. Only the final product identity, mask output HDUs and references to the source product ID are remapped. Each complete original detector ASDF is retained as text evidence, together with the explicit source/output image-plane map and the recipe snapshot. The final container is self-contained: the saved reader needs neither the source detector artifacts nor raw files. Metadata is bounded; there is no all-detector pixel cube. The existing 64 KiB copy buffer is reused.

The independent reader entry point is

`pycavi_bpm_linearity_assembly.read_product`.

It validates the common schema and checks every joined measurement and binding against the embedded original records, including the permitted identity/HDU remapping. Native FITS readback checks layout before exclusive publication; a separate independent comparison checks every final padded image payload.

Historical calibration locations need not be rewritten in their original ASDF records. Explicit recipe `calibration_bindings` declare the original location, a copy below the project's calibration root, and the preserved SHA256. Only master/BPM locations are rebound; HDUs, asset IDs, raw paths and scientific settings remain fixed. Admission compares the declared digest with the original archive; batch verification checks the actual copied bytes. Native preparation embeds both the unchanged source graph and these declarations. Its saved reader checks that the context changed only the declared locations. Project preparation copies and verifies calibration bytes without decoding pixels or running a reduction. A conflicting fingerprint is refused, never adopted as a new baseline.

`read_manifest` verifies joined ASDF hashes and source/detector mapping after relocation, without FITS pixels or native commands. FITS integrity rechecking is explicitly false for this metadata reader; independent image comparison remains necessary. The C publication path passed its synthetic Mac gate at revision `6f3eaaf`. The CCD65 saved-product inspection and Viewer plots have since passed owner inspection; extension to the full detector cohort remains pending.

Native recipe: structure and ownership

The native entry point is `mefpp2/ins/mef/recipes/mefpp_bpm_linearity_recipe.c`. Like `mefpp_bias_recipe.c`, its visible order is initialize/start, prepare/admit/begin, serial detector loop, verify/publish/complete, and one failure/report/destruction path. Both use `mef_recipe_run`; the recipe owns no all-detector image cube. There are no new estimators or changed populations in this migration.

The authoring TOML selects one `native_recipe` executable and exact source revision. The assembly table selects the reference detector; child executables are not configured in this form. The original per-stage form remains supported for retained records. Mixing the two execution forms is refused. Project TOML remains the sole owner of shared directory roots.

Pycavi admits historical ASDF dependencies, calibration-copy fingerprints, detector/processing controls and reference ROI names before processing. It freezes explicit native bindings in orchestration; staged archive metadata remains below reduced/work until C copies it into the final product tree. Those bindings contain the existing native argument contracts, metadata-copy declarations and unique source paths with SHA256 identities. They do not introduce alternative scientific settings. Native execution is one command:

```
mefpp_bpm_linearity_recipe cal bpm-linearity
--config RECIPE.toml --bindings NATIVE.toml
```

Given admitted bindings, this command runs without Pycavi, Python or a shell. Historical archive-graph resolution remains a metadata-authoring step.

`prepare` reads the authoring recipe/project/bindings and expands only the single captured start timestamp. The existing BPM handoff retains whole-second precision derived from that same instant; it samples no second clock. `admit` checks roots, detector order, master/prior consistency, fresh destinations, niceness and source fingerprints. `begin` creates output/work/log directories and copies frozen metadata, without copying raw images. `process` calls the existing native preparation and BPM functions in process. Each service releases its image owners before the next stage/detector. The same master and prior remain fixed. `verify` requires complete detector artifacts. `publish` reuses the final FITS copier and native QC join described above, then verifies source bytes again. No scientific result is recalculated during assembly or saved reading.

Shared inputs are hashed once before and once after the complete batch; between stages only device/inode/size/mode/nanosecond-mtime identity is checked. The final content check also detects restored-timestamp edits. SHA256 uses the standard FIPS 180-4 construction with a 64 KiB read buffer. On macOS the system CommonCrypto incremental SHA256 API is used; other hosts retain the portable implementation, with the same known-answer checks. This avoids replacing the Mac hash provider with a scalar C implementation. Exact output hashes are stored in `native-recipe.toml`. A successful receipt also records measurement-only state and input preservation; command success additionally requires zero exit. Failed work remains available. Pycavi independently reads native ASDF and checks its settings/bindings before writing the familiar combined manifest; it does not complete or publish QC.

Native recipe source map

Stable comment landmarks connect the entry point and C helpers to this section. Helpers live under `mefpp2/ins/mef/common/`.

1. `BPM-LINEARITY-ADMISSION`.

   Sources: `mef_bpm_linearity_config.c`, `mef_bpm_linearity_inputs.c` and `mef_bpm_linearity_bindings.c`. They admit roots, detector bindings, frozen dependencies and raw/calibration transport paths. Preparation and BPM must use the same ordered flat paths/HDUs and native configuration.

2. `BPM-LINEARITY-DETECTOR-LOOP`.

   The entry point calls `mef_bpm_linearity_process.c` serially, keeping preparation before candidate characterization.

3. `BPM-LINEARITY-MEASUREMENTS`.

   The process helper calls `mef_linearity_prepare_command`. Row correction and statistics remain in the preparation/files/pairs kernels mapped earlier.

4. `BPM-LINEARITY-CANDIDATE`.

   The same helper calls `mef_bpm_command`. Fitting remains in `mef_cal_bpm.c` and `detector_response.c`.

5. `BPM-LINEARITY-PUBLICATION`.

   The entry point calls `mef_bpm_linearity_products.c`: final MEF assembly and the unchanged native ASDF join.

6. `BPM-LINEARITY-PRESERVATION`.

   The products helper and `mef_bpm_linearity_hash.c` check streaming identities and publish the exclusive completion receipt.

The Python metadata/call boundary is `pycavi_bpm_linearity_native_recipe.py`; it contains no detector-stage execution loop. The focused Mac qualifier builds only the coordinator and runs one small synthetic complete recipe with independent ASDF/schema/value checks. Local numerical/FITS checks with a labelled test writer do not qualify native ASDF or observational behavior. Those gates remain mandatory before selection.